
DAWN-SRW: A Computational Model of Learned State Transitions

Seungho Choi¹

Abstract

Language-model blocks evolve representation states through relational interaction and subsequent transformation. DAWN-SRW asks whether these otherwise implicit mechanisms can instead be organized explicitly around reusable learned operations, continuous state-conditioned applicability, and their composition. It realizes this organization with rank-1 read-write (RW) primitives. Each current representation state determines which primitives participate and to what continuous degree. Their normalized composition constructs the query, key, and value representations used by attention, or directly produces the post-attention residual update. This organization admits a program-like interpretation as continuous conditional execution over learned operators controlled by the current representation. The goal is not to assume that those operations are semantically human-readable, but to make transition-producing computation architecturally explicit enough to identify and causally intervene on its constituent operations. In the evaluated v4172 realization, learned operation-address mappings implement this control by comparing state-derived queries with bilinear addresses of primitive read/write directions. At 393.8M parameters and 39.999B tokens, DAWN-SRW retains performance against a matched dense Transformer: C4 cross-entropies are 2.9402 versus 2.9359, and six-task zero-shot means are 0.4629 versus 0.4597. A descriptive 1.272B evaluation reaches 2.8359 cross-entropy and a 0.4858 zero-shot mean. A discovery-frozen 4,096-site Q/K-centered operator circuit passes suppression, four matched-control families, and exact restoration on independent validation and held-out indirect-object-identification inputs. The model also exhibits exact input-conditioned functional support, although execution remains dense. These results support explicit state-conditioned RW composition as vi-

able and causally analyzable, without claiming benchmark superiority or hardware efficiency.

1. Introduction

Language-model blocks repeatedly transform representation states across depth. Let X_ℓ denote the sequence representation at layer ℓ ; at a coarse level, a block implements

$$X_\ell \longrightarrow X_{\ell+1}.$$

We use this state-evolution view only as an organizing observation, not as a contribution.

In conventional Transformers, the mechanisms producing these state changes remain largely implicit in dense parameterized transformations. Attention and feed-forward layers expose module boundaries, but they do not natively factor transition-producing computation into reusable operations, explicit state-dependent applicability, and the direct composition of those applicable contributions.

From this perspective, transition-producing computation can be studied as a learned conditional-operator problem: the model learns reusable transformations, the representation-dependent conditions under which they participate, and how their contributions compose. We use this as a research perspective for exposing and organizing learned computation, rather than as an equivalent characterization of all language-model computation.

We therefore ask whether transition-producing computation can be decomposed architecturally into three explicit parts: reusable learned operations, state-dependent applicability, and composition. Such a representation should expose which operations exist, which apply to the current state and to what degree, and how their contributions form the computation used in the next update. This decomposition does not require individual operations to correspond to human-readable concepts; semantic interpretation remains a subsequent empirical question.

This motivates the following architectural question:

Can transition-producing language-model computation be explicitly organized around reusable learned operations, continuous state-dependent applicability, and their composition?

¹Independent Researcher. Correspondence to: Seungho Choi <madst0614@gmail.com>.

Rather than recovering such structure post hoc from an already-trained dense model, DAWN-SRW makes it explicit by construction and tests whether it can support end-to-end language modeling.

DAWN-SRW realizes this organization by taking a rank-1 read-write transformation as its basic RW computational primitive. For a token state x ,

$$\mathcal{O}_i(x) = \langle x, r_i \rangle w_i, \quad (1)$$

$$F(x) = \frac{1}{Z(x)} \sum_i c_i(x) \mathcal{O}_i(x), \quad c_i(x) \in [0, 1]. \quad (2)$$

The read direction r_i determines what is extracted from the current representation, the write direction w_i determines the direction of the primitive’s output contribution, and $c_i(x)$ determines whether and to what degree that primitive applies. Thus, DAWN-SRW parameterizes transition-producing computation as explicit conditional composition of reusable RW primitives.

The control abstraction is also explicit:

$$x \mapsto C_a(x) = (c_{a,1}(x), \dots, c_{a,N_a}(x)). \quad (3)$$

Together, these components admit an operational, program-like interpretation: the representation supplies the evolving computational state, $C_a(x)$ controls explicit learned operations, and composition produces the computation used in the next update. Classical operational accounts describe computation through transitions between configurations (Plotkin, 1981); we use this perspective as a conceptual reference. Compared with familiar binary predicate-controlled execution, DAWN-SRW represents operator applicability continuously and composes the contributions of applicable operations.

In the evaluated v4172 realization, the QK-, V-, and RST-specific RW banks persist across depth and share an address coordinate convention, while each successive representation state recomputes applicability over the corresponding bank. The model therefore conditionally reuses persistent type-specific RW banks rather than selecting unrelated layer-local units. This profile is realized through

$$x \mapsto (q_a(x), \tau_a(x)) \mapsto c_{a,i}(x),$$

using primitive addresses $k_{a,i}$. The operation-address space implements this applicability control; it does not execute RW computation. Each RW primitive defines a candidate computational contribution, while state-conditioned applicability determines whether and to what degree that contribution participates in the composed output.

This construction builds on the read-write view of dense feed-forward computation, where first-layer directions detect input patterns and second-layer directions contribute

outputs (Geva et al., 2021). That analysis reveals functional directions inside an existing dense layer. DAWN-SRW instead turns RW factorization into an architectural constraint: reusable RW units are executable RW primitives whose state-dependent contributions are directly controlled and observable.

The same formulation produces the query, key, and value states used by causal self-attention and the direct update applied to the post-attention residual state. Q and K produce separate outputs from one shared QK pool; V and the post-attention residual transition (RST) use separate pools. Operator pools and applicability projections are reused across depth, while normalization and attention output projections remain layer-specific.

The empirical study follows a fixed evidence order. We first compare a 393.8M parameter DAWN-SRW checkpoint with a parameter- and token-matched dense Transformer on C4 and six zero-shot tasks. We then report descriptive 1.272B results, measure exact input-conditioned operator support, and localize a discovery-only indirect object identification (IOI) circuit. The circuit is frozen before confirmatory data are opened, then tested with suppression, matched controls, and exact restoration on independent validation and fully held-out test inputs. Finally, we report three fail-closed boundaries: RAVEL localization, ARC causal validation, and 1.3B IOI localization. Efficiency is not a headline result: the current implementation scores and applies full operator pools densely.

Our contributions are:

1. **Explicit state-conditioned operator organization.** We organize transition-producing computation around reusable rank-1 RW operations, continuous state-conditioned applicability, and normalized composition into the interfaces used by attention or a direct residual-state update. In the evaluated v4172 realization, applicability is implemented through learned spherical operation-address mappings, yielding exact state-dependent support.
2. **End-to-end viability and operator-level causal intervention.** We show end-to-end language-model viability at 400M scale, report a descriptive 1.3B evaluation, characterize exact input-conditioned operator support in the primary 400M model, and show on IOI that exposed operator contributions can serve as causal intervention targets. A discovery-frozen IOI circuit is confirmed by suppression, matched controls, and exact restoration on independent validation and fully held-out inputs.

2. Related Work

2.1. Conditional computation

Sparingly-gated mixtures of experts activate a small subset of large expert networks (Shazeer et al., 2017), and Switch Transformers scale this pattern with simple routing (Fedus et al., 2022). PEER uses product-key retrieval over more than a million tiny experts (He, 2024); OmniMoE extends fine-grained conditional computation to vector-level Atomic Experts (Shi et al., 2026). Thus fine-grained reusable units and state-dependent retrieval predate DAWN-SRW. DAWN-SRW instead uses an explicit state-conditioned operator organization as the base parameterization of transition-producing language-model computation and evaluates the resulting operator contributions as causal intervention targets. Current execution remains a dense address scan and dense, chunked RW application rather than an indexed sparse kernel.

2.2. Neural programming and programming interpretations

Neural Programmer-Interpreters represent and execute programs from supervised traces (Reed & de Freitas, 2016), while Neural Programmer differentially selects operations and data (Neelakantan et al., 2016). Fast Weight Programmers show an algebraic connection: linearized self-attention is equivalent to fast-weight programming, with additive outer products interpreted as elementary instructions (Schlag et al., 2021). DAWN-SRW neither induces a symbolic program nor defines a formal semantics; its program-like interpretation concerns explicit learned RW operations, continuous applicability, and composition inside a language-model block.

2.3. Read-write primitives and memory views

Transformer blocks combine attention and position-wise feed-forward networks over a residual stream (Vaswani et al., 2017). Geva et al. interpret the feed-forward layers as key-value memories, with read-side pattern detection and write-side output directions (Geva et al., 2021). Product Key Memory factorizes retrieval keys (Lample et al., 2019), and later work scales trainable key-value memory layers (Berges et al., 2024). MoRAM further treats rank-1 read/write factors as atomic associative-memory experts and uses the read-side response itself to determine relevance (Lu et al., 2026). DAWN-SRW differs in using RW primitives as the language model’s base computational substrate and evaluating applicability through a separate learned address representation.

2.4. Mechanistic analysis and causal intervention

Mechanistic interpretability studies model behavior through identifiable components and causal interventions. Work on IOI isolates an attention-head circuit and evaluates faithfulness, completeness, and minimality (Wang et al., 2023). Activation patching and scalable approximations localize behavior to components (Kramár et al., 2024). RAVEL emphasizes that neuron-level representations may be entangled and evaluates methods against causal criteria (Huang et al., 2024). Our analysis treats a site as a layer, computation type, and RW-operator identity; freezes localization before confirmation; and uses direct numerator suppression, matched random circuits, and exact same-example restoration. Explicit RW units make the intervention target precise, but do not guarantee a human-readable interpretation.

3. DAWN-SRW

3.1. State Evolution as the Starting Abstraction

Let

$$X_\ell = (x_{\ell,1}, \dots, x_{\ell,T}) \in \mathbb{R}^{T \times D}$$

denote the sequence representation at depth ℓ . At a coarse level, a Transformer-like block evolves this state through a relation-dependent attention update followed by a transformation of the contextualized representation:

$$Y_\ell = X_\ell + A_\ell(X_\ell), \quad (4)$$

$$X_{\ell+1} = Y_\ell + R_\ell(Y_\ell). \quad (5)$$

This decomposition only fixes the computational abstraction and notation. DAWN-SRW concerns how the transformations producing these updates are parameterized.

3.2. Explicit State-Conditioned Operator Parameterization

DAWN-SRW parameterizes transition-producing computation as state-conditioned operator composition. This higher-level construction separates three components: reusable RW primitives, a state-dependent applicability profile that supplies control, and normalized composition of their contributions. For a token state $x \in \mathcal{S} = \mathbb{R}^D$, let $a \in \{Q, K, V, \text{RST}\}$ index the computation type and define

$$F_a : \mathcal{S} \rightarrow \mathbb{R}^D. \quad (6)$$

The Q, K, and V outputs form the state-conditioned interfaces through which relational interaction is computed, whereas F_{RST} directly specifies a residual-state update. Each composed output is explicitly decomposed into contributions from reusable RW primitives under state-dependent control. Control is therefore one component of state-conditioned operator composition rather than the whole abstraction. The operation-address space is the evaluated

implementation of this control mechanism rather than the paper’s core hypothesis.

3.3. Read-Write Computational Primitives

Each primitive owns learned read and write parameters. With $\text{unit}(z) = z / \max(\|z\|_2, \epsilon)$, define normalized directions $\bar{r}_{a,i} = \text{unit}(r_{a,i})$ and $\bar{w}_{a,i} = \text{unit}(w_{a,i})$. The primitive executes

$$\mathcal{O}_{a,i}(x) = \langle x, \bar{r}_{a,i} \rangle \bar{w}_{a,i} = \bar{w}_{a,i} \bar{r}_{a,i}^\top x. \quad (7)$$

This is an explicit rank-1 RW primitive. It reads a scalar from the current state and produces a directed output contribution. The read direction specifies the extracted scalar, while the write direction specifies the direction of that contribution. For Q, K, and V, these contributions compose the interfaces used by attention; for RST, they compose a direct residual-state update. Unlike a static retrieved value, an RW primitive remains an input-dependent transformation, with its output varying with the current state through the read response. The state-to-state transition is completed only when the composed output participates in the attention-mediated or residual update.

3.4. State-Conditioned Applicability

At the abstract level, control is represented by an applicability profile over reusable RW primitives. For each computation type $a \in \{Q, K, V, \text{RST}\}$, the current state induces a profile over all N_a primitives,

$$C_a(x) = (c_{a,1}(x), \dots, c_{a,N_a}(x)), \quad (8)$$

whose coordinates specify which primitives participate and to what degree, rather than assigning the state to a single discrete branch. The evaluated v4172 realization has exact zero support outside learned boundaries and continuously positive applicability inside them. This is not an algebraic decomposition of the representation vector; its coordinates control the contributions of the corresponding RW primitives composed below.

In v4172, normalized coordinates $q_a(x)$ and $k_{a,i}$ compare the current state with primitive i ’s learned address; Section 3.6 gives their construction. A separate direct projection produces the state-dependent DirectTau boundary:

$$\tau_a(x) = -1 + 2\sigma(xu_a + \beta_a), \quad (9)$$

$$\rho_{a,i}(x) = q_a(x)^\top k_{a,i}, \quad (10)$$

$$m_{a,i}(x) = \rho_{a,i}(x) - \tau_a(x). \quad (11)$$

The evaluated linear-angular rule defines primitive applicability as

$$c_{a,i}(x) = \text{clip}\left(\frac{m_{a,i}(x)}{\max(1 - \tau_a(x), 10^{-4})}, 0, 1\right). \quad (12)$$

We call $c_{a,i}(x) \in [0, 1]$ the *continuous conditional applicability* of primitive i at state x . Its exact state-dependent support is

$$\mathcal{I}_a(x) = \{i : \rho_{a,i}(x) > \tau_a(x)\} = \{i : c_{a,i}(x) > 0\}. \quad (13)$$

Outside this set, a primitive contributes exactly zero; inside it, $0 < c_{a,i}(x) \leq 1$. A primitive’s contribution therefore enters continuously from zero at the learned boundary. This combines state-dependent sparse support with a continuous-valued degree of applicability rather than merely assigning a dense soft mixture over every primitive.

3.5. State-Conditioned Operator Composition

Applicable primitives compose into the output $F_a(x)$:

$$Z_a(x) = \left[\max\left(\sum_i c_{a,i}(x), 1\right) \right]^{p_a}, \quad (14)$$

$$s_a = \sqrt{D/L},$$

$$F_a(x) = \frac{s_a}{Z_a(x)} \sum_i c_{a,i}(x) \mathcal{O}_{a,i}(x). \quad (15)$$

Equivalently,

$$F_a(x) = A_a(x)x, \quad (16)$$

$$A_a(x) = \frac{s_a}{Z_a(x)} \sum_i c_{a,i}(x) \bar{w}_{a,i} \bar{r}_{a,i}^\top.$$

This form makes the architectural hypothesis explicit. The model does not apply one fixed dense transformation to every state. Instead, the current representation determines continuous applicability coefficients, which construct a state-dependent transformation $A_a(x)$ from reusable rank-1 RW primitives. The resulting computation is both input-conditioned and explicitly decomposed into identifiable operator contributions.

“Continuous” refers to the value of operator applicability, not to continuous-time dynamics or a continuous-depth model; layer evolution remains discrete. The scalar mapping from normalized margin to applicability is continuous and piecewise linear, with nondifferentiable support boundaries.

The denominator powers are $p_Q = p_K = 0.5$, $p_V = 1.0$, and $p_{\text{RST}} = 1.2$. The fixed composition scale is $s_a = \sqrt{D/L}$, with no learned pool-strength parameter.

3.6. Operation-Address Space

The operation-address space $\mathcal{A} = \mathbb{R}^R$ is where primitive applicability is evaluated, not where RW computation itself is executed. This addressing construction is one evaluated realization of the state-conditioned applicability abstraction above. The present experiments do not establish that this particular realization is necessary or preferable to alternative

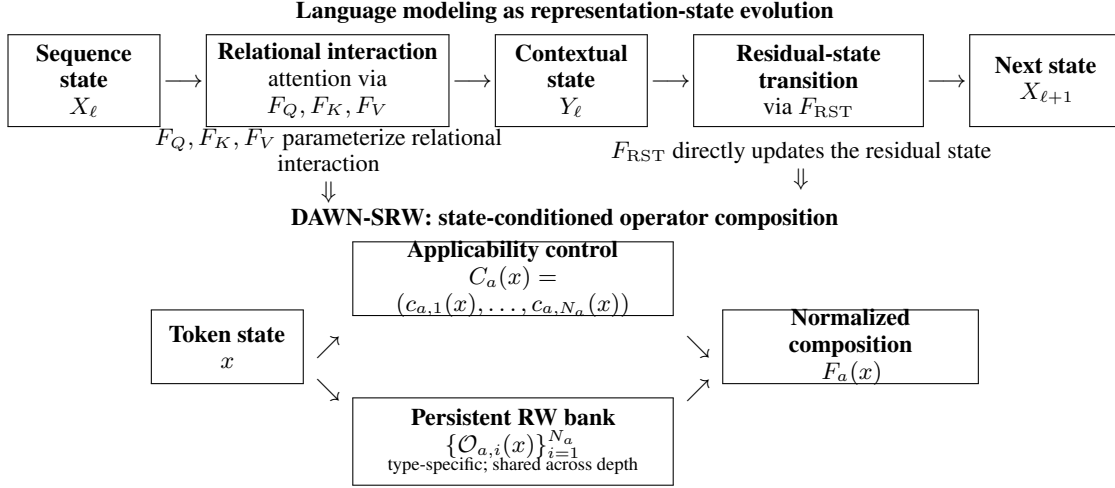


Figure 1. The two-level construction. Top: a language-model block evolves a sequence representation through relational interaction and a subsequent residual-state transition. Bottom: x separately determines the applicability profile and input-dependent responses of a persistent, type-specific RW bank shared across depth; the two meet in normalized composition $F_a(x)$. In v4172, $q_a(x)$, $\tau_a(x)$, $k_{a,i}$ implement applicability in address space; RW execution remains in the full D -dimensional representation space.

state-conditioned control mechanisms. In v4172, $R = 256$, and a normalized query is a direct projection of the current input state:

$$q_a(x) = \text{unit}(xP_a + b_a) \in \mathcal{A}. \quad (17)$$

A shared probe pair $P_r, P_w \in \mathbb{R}^{D \times R}$ maps all operator pools' functional read and write directions into address features. The operator address is

$$\begin{aligned} u_{a,i} &= \text{unit}(\bar{r}_{a,i}^\top P_r), & v_{a,i} &= \text{unit}(\bar{w}_{a,i}^\top P_w), \\ k_{a,i} &= \text{unit}(u_{a,i} \odot v_{a,i}) \in \mathcal{A}. \end{aligned} \quad (18)$$

This generalized coordinate-wise bilinear mapping makes address identity a learned function of what a primitive reads and where it writes, rather than an independent expert ID. It is the addressing realization used to compute the applicability coefficients in Equation 12, not the paper's definition of a state transition. Shared probes align QK, V, and RST addresses to one coordinate convention, while their operator rows remain in QK-, V-, and RST-specific pools.

Because both $q_a(x)$ and $k_{a,i}$ are unit-normalized, they lie on the address sphere $\mathbb{S}^{R-1}$. Let $\theta_{a,i}(x) = \arccos(q_a(x)^\top k_{a,i})$ and $\theta_a(x) = \arccos \tau_a(x)$. The active support is therefore

$$\mathcal{I}_a(x) = \{i : \theta_{a,i}(x) < \theta_a(x)\},$$

a spherical cap centered at the query $q_a(x)$ with state-dependent angular radius $\theta_a(x)$. Geometrically, $q_a(x)$ specifies the center of the applicability query, $\tau_a(x)$ specifies its angular support boundary, and $c_{a,i}(x)$ specifies the continuous degree to which primitive i applies. When the numerical

guard in Equation 12 is inactive, an active primitive has

$$c_{a,i}(x) = \frac{\cos \theta_{a,i}(x) - \tau_a(x)}{1 - \tau_a(x)},$$

which rises continuously from zero at the cap boundary to one at its center. Equivalently, $c_{a,i}(x)$ is a normalized continuous depth within the cap. Equation 12 is the exact implementation, retaining the 10^{-4} denominator guard and clipping.

3.7. Attention-Mediated and Residual Transitions

For a sequence state X_ℓ , layer-specific normalization gives $\tilde{X}_\ell = \text{LN}_\ell^{(1)}(X_\ell)$. Three state-conditioned operator compositions produce the attention interfaces:

$$Q_\ell = F_Q(\tilde{X}_\ell),$$

$$K_\ell = F_K(\tilde{X}_\ell),$$

$$V_\ell = F_V(\tilde{X}_\ell),$$

$$A_\ell = \text{softmax}\left(\frac{Q_\ell K_\ell^\top}{\sqrt{d_h}} + M_{\text{causal}}\right),$$

$$\Delta X_{\text{attn},\ell} = A_\ell V_\ell W_\ell^O, \quad (19)$$

$$Y_\ell = X_\ell + \Delta X_{\text{attn},\ell}. \quad (20)$$

Q and K produce separate outputs from the same 4,600-row QK pool; V produces an output from a separate 14,500-row pool. After attention, a second layer-specific normalization $\tilde{Y}_\ell = \text{LN}_\ell^{(2)}(Y_\ell)$ is evaluated anew using the 29,300-row RST operator pool:

$$X_{\ell+1} = Y_\ell + F_{\text{RST}}(\tilde{Y}_\ell). \quad (21)$$

The block thus applies an attention-mediated state transition followed by a direct residual-state transition. RST applicability depends on the contextualized post-attention state, not the pre-attention query.

3.8. Cross-Depth Operator Reuse

The QK, V, and RST read/write pools; the two RW-to-address probes; the Q/K/V query projection; the RST query projection; and their DirectTau projections are shared across all L layers. Q and K share read/write rows but retain separate query slices, tau values, applicability coefficients, and outputs. The layer-specific parameters are two LayerNorms and the attention output projection W_ℓ^O . Positional and token embeddings and the final normalization are model-level parameters. Consequently, DAWN-SRW reuses RW primitives across depth without claiming complete block sharing.

3.9. Implemented Addressing and Compute Boundary

The implementation shards operator rows across the model mesh and processes them in dense chunks. The 400M configuration uses one QK chunk, one V chunk, and four RST chunks. Every query scores the full local address pool; every operator row participates in dense read and write contractions before cross-shard reduction. Exact zeros in Equation 12 therefore describe *functional* support, not an indexed sparse kernel.

The evaluated architecture also keeps RW execution in the full residual dimension D . The low-dimensional space is only an applicability-address space. An executor could omit exactly zero RW terms without changing the mathematical output, but that conditional execution opportunity is not implemented or timed here.

4. Experimental Setup

4.1. Models and training

Table 1 summarizes the evaluated checkpoints. The primary DAWN-SRW model has 393,800,708 measured parameters, operation-address dimension $R = 256$, and 48,400 operator rows across QK, V, and RST. It was trained for 39,999,504,384 packed C4 tokens with sequence length 512 and global training batch 1,024. Both dropout rates are zero.

4.2. Matched 400M dense control

The dense control has 393,896,960 measured parameters, width 1,280, feed-forward width 5,120, 18 layers, and 20 heads. It uses the same tokenizer, vocabulary, sequence length, training-token count, seed, batch size, mesh, learning rate, warmup ratio, weight decay, and zero dropout as the primary model. The parameter difference is 96,252, or

0.0244% of the dense model. This comparison tests performance retention under matched scale and training exposure; it is not a formal equivalence test.

4.3. Evaluation protocols

The C4 protocol evaluates 610 complete global batches from the fixed 10M-token capped packed-validation view: 19,520 sequences and 9,974,720 valid next-token targets. Cross-entropy, perplexity, and next-token accuracy use float32 scoring. The zero-shot protocol uses `lm-eval` 0.4.2 with no few-shot examples and no task limit on LAMBADA, HellaSwag, PIQA, ARC-Easy, ARC-Challenge, and WinoGrande. The primary metric is raw accuracy for LAMBADA and WinoGrande and normalized accuracy for the other four tasks. The mean is an unweighted mean of these six metrics.

4.4. Mechanistic localization and frozen intervention protocol

We define a site by layer, computation type, token position, and global operator identity. IOI discovery uses 119 independent paired-correct units. Sites are ranked by absolute discovery mean contribution importance. We choose the smallest audited prefix that reaches at least 0.70 cumulative absolute importance and 0.95 split top- k overlap. The resulting 4,096-site Q/K/V/RST/layer composition and intervention specification are frozen before validation.

The primary causal score is the summed-log-probability margin between the correct indirect object and the source name. Circuit-wide suppression zeros only the selected operators' execution numerator at target positions while retaining the full production denominator. Four control families each contain 100 frozen-size circuits that exclude circuit sites: equal-count random, same-layer random, activation-matched, and route-frequency-matched across Q/K/V/RST. Exact restoration reinserts the same-example selected numerator after suppression, again under the production denominator. Validation uses 123 independent paired-correct units; the held-out test accessor is opened only after the validation result and unchanged specification are finalized. We report 95% bootstrap confidence intervals and two-sided paired permutation tests where specified.

5. Results

5.1. Language-model performance retention at 400M

Table 2 reports the locked C4 evaluation. DAWN-SRW differs from the dense control by +0.004318918 cross-entropy, +0.081537182 perplexity, and -0.002188432 accuracy (-0.2188 percentage point). These small measured gaps support performance retention, not superiority or formal equivalence.

Table 1. Model and training specification. The 1.3B row is descriptive and does not form a controlled scaling curve with the 400M rows.

Model	Parameters	Tokens	D	Layers	Heads	QK / V / RST rows	Checkpoint
DAWN-SRW 400M	393,800,708	39,999,504,384	2,304	18	36	4,600 / 14,500 / 29,300	76,293
Dense Transformer 400M	393,896,960	39,999,504,384	1,280	18	20	—	76,293
DAWN-SRW 1.3B	1,272,038,404	19,999,981,568	4,096	24	32	8,534 / 26,904 / 54,352	87,193

Table 2. Packed C4 validation on the identical 9,974,720 target tokens.

Model	CE	PPL	Accuracy
DAWN-SRW 400M	2.940213	18.919873	0.424912
Dense 400M	2.935894	18.838336	0.427100
DAWN-SRW 1.3B	2.835879	17.045382	0.437099

Table 3 gives task-level zero-shot results with standard errors. The matched 400M means are 0.462872 and 0.459652, a difference of $+0.003220495$ ($+0.3220$ percentage point). Task differences vary in sign, so the mean does not establish benchmark superiority. LAMBADA perplexities are 24.0774 ± 0.8005 for DAWN-SRW 400M, 24.0042 ± 0.8073 for dense 400M, and 17.6358 ± 0.5390 for DAWN-SRW 1.3B.

5.2. Descriptive larger-scale evaluation at 1.3B

The 1.272B checkpoint reaches 2.835879 C4 cross-entropy, 17.045382 perplexity, 0.437099 token accuracy, and a 0.485844 six-task mean. It was trained for 19.999B tokens, about half the 400M training-token count, and no completed matched dense 1.3B result is admitted to this study. These results therefore describe a larger trained DAWN-SRW model; they are neither a controlled scaling curve nor a dense-model comparison.

5.3. Input-conditioned RW operator support

At all 9,994,240 packed-C4 positions, Q, K, V, and RST each have strict-subset operator support at positive selection margin (Table 4). Mean exact support is 487.896 Q operators, 386.722 K operators, 812.388 in the shared-pool Q/K union, 1,649.994 V operators, and 2,017.703 RST operators. This is exact input-conditioned functional support under Equation 12.

The support does not make the present kernel efficient. Static forward accounting at batch 1 and sequence length 512 is 4.741260116 TFLOPs for current dense DAWN-SRW, 0.818230250 TFLOPs for dense addressing plus exact-support-only RW application, and 0.386547057 TFLOPs for the matched dense Transformer. The exact-support opportunity is 17.2577% of current DAWN-SRW accounting but still 2.1168 times the dense Transformer (Figure 2). It is a mathematical conditional-RW opportunity, not measured latency, energy, or hardware speedup.

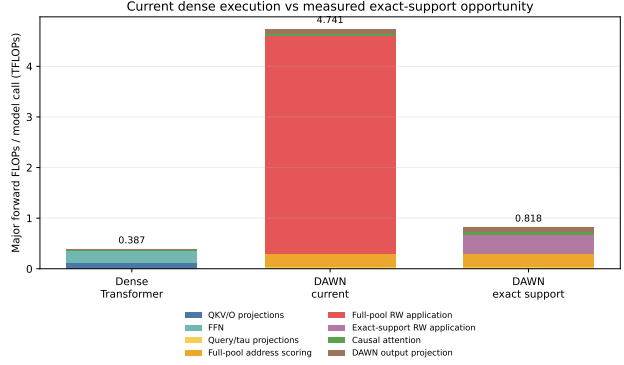


Figure 2. Static backbone accounting at batch 1 and sequence length 512. Address scoring remains dense in the exact-support estimate. Current execution also applies all full-dimensional RW rows. No production latency claim is made.

5.4. IOI circuit localization

Discovery produces split-half rank stability 0.9139788 over 456,697 ranked sites. The smallest prefix passing both frozen discovery gates contains 4,096 sites and 0.719249 cumulative absolute importance. It spans all 18 layers and contains 1,744 Q sites, 2,350 K sites, two V sites, and no RST sites. Split top- k overlap is 0.974854 (Jaccard 0.950941). These statistics identify a Q/K-centered candidate circuit; localization alone is not a causal claim.

5.5. Validation and held-out causal intervention

Table 5 shows confirmatory results. On validation, suppression moves the mean margin from 3.465456 to -4.399331 , a drop of 7.864787 (95% CI [7.490080, 8.241263]), and accuracy falls from 1.000000 to 0.008130. The frozen effect exceeds every replicate in all four 100-circuit control families. The most competitive control is activation-matched, with mean drop 5.400490; the frozen-minus-control difference is 2.464297 (95% CI [2.076200, 2.832697], paired permutation $p = 0.00049975$). Exact restoration returns accuracy to 0.991870 and recovers 0.999661 of the suppression effect (95% CI [0.998655, 1.000584]).

The fully held-out test reuses the unchanged site identity, scoring rule, intervention, controls, seeds, and thresholds. Suppression moves margin from 3.254720 to -4.155613 , a drop of 7.410332 (95% CI [7.016485, 7.818472]), while accuracy falls to 0.008772. Again the frozen effect exceeds every replicate in all four control families. The activation-

Table 3. Zero-shot accuracy. Parentheses contain standard errors. HS, AE, and AC denote HellaSwag, ARC-Easy, and ARC-Challenge.

Model	LAMBADA	HS	PIQA	AE	AC	WinoGrande	Mean
DAWN-SRW 400M	0.352028 (0.006654)	0.458375 (0.004972)	0.712187 (0.010563)	0.484007 (0.010255)	0.256826 (0.012767)	0.513812 (0.014047)	0.462872
Dense 400M	0.353192 (0.006659)	0.457080 (0.004971)	0.698041 (0.010712)	0.475589 (0.010248)	0.249147 (0.012639)	0.524862 (0.014035)	0.459652
DAWN-SRW 1.3B	0.395886 (0.006813)	0.510157 (0.004989)	0.723069 (0.010440)	0.509680 (0.010258)	0.268771 (0.012955)	0.507498 (0.014051)	0.485844

Table 4. Exact functional support at selection margin > 0 . Q/K union counts an operator active for Q or K at the same layer and token.

Computation	Pool	Mean active	Pool fraction
Q	4,600	487.896	10.6064%
K	4,600	386.722	8.4070%
Q/K union	4,600	812.388	17.6606%
V	14,500	1,649.994	11.3793%
RST	29,300	2,017.703	6.8864%

matched mean drop is 5.472579; the frozen-minus-control difference is 1.937754 (95% CI [1.564818, 2.319706], $p = 0.00049975$). Restoration recovers 0.998406 of the effect (95% CI [0.997324, 0.999385]) and restores accuracy to 1.000000.

Together, suppression, matched-control separation, and exact restoration support the claim that this discovery-frozen Q/K-centered operator circuit is causally necessary for the evaluated IOI behavior on independent validation and fully held-out inputs. “Necessary” is intervention-specific: it does not mean the sites form a unique or human-readable algorithm.

5.6. Negative and fail-closed results

The same workflow produces informative negative boundaries (Table 6). RAVEL passes the behavioral gate but fails variable-localization stability, so no circuit is frozen and no confirmatory split is opened. ARC discovery freezes a stable 4,096-site Q/K circuit, but validation suppression has the wrong sign and fails control separation; restoration passes, and test remains closed. For 1.3B IOI, behavioral eligibility and rank stability pass, but the largest allowed 4,096-site prefix reaches only 0.635144 cumulative importance, below the preregistered 0.70 threshold. No circuit is frozen and no validation or test is opened.

6. Discussion

6.1. What the results establish

The evaluated model shows that a language model can construct attention and post-attention residual updates by state-conditioned composition of explicit RW primitives. The matched 400M comparison shows performance retention under the measured protocols. The support profile shows that applicability has exact input-conditioned zeros, and the IOI study shows that operator sites can form a discovery-

frozen circuit whose suppression and restoration causally control behavior on held-out data.

These claims are deliberately narrower than a theory of language reasoning as discrete programs. The causal result is strong for the specified IOI task and intervention, not universal across tasks or scales. RAVEL, ARC, and 1.3B IOI demonstrate that explicit units do not make successful localization automatic.

6.2. Operational Interpretation and Computational Interface

This organization admits a program-like interpretation as continuous conditional execution over learned operators. Under this view, learning transition-producing computation includes learning not only operator functions, but also the representation-dependent conditions under which those operators participate and how their contributions compose. The representation supplies the current computational state, the applicability profile controls explicit learned operations, and normalized composition produces the computation used by the subsequent attention or residual update. This is an architectural interpretation of exposed computation, not a formal operational semantics or an assumption of semantic interpretability. A learned primitive may remain polysemantic, and a transition may depend on a large distributed set of primitives. DAWN-SRW does not claim that such computation is therefore easy for a human to understand or manipulate; it exposes the constituent operations, their state-dependent applicability, their executed contributions, and their composition.

Cross-depth sharing makes this computational interface persistent: the same QK-, V-, or RST-specific primitive can be revisited at different depths as the evolving representation changes its applicability. An analysis can therefore trace when the same identified operation participates and how strongly it contributes across depth, rather than compare unrelated layer-local units. In v4172, the type-specific pools remain separate, but their addresses share a learned coordinate convention; this does not imply a single pool in which all operator types compete.

This provides an analysis interface that is absent when the same computation is represented only as an implicit dense transformation. One may inspect which primitives participate, suppress or restore their contributions, trace how applicability changes with the representation state, and test

Table 5. Frozen IOI circuit results. Accuracy arrows show intact to suppressed. Recovery is the fraction of the suppression effect recovered by exact same-example numerator restoration.

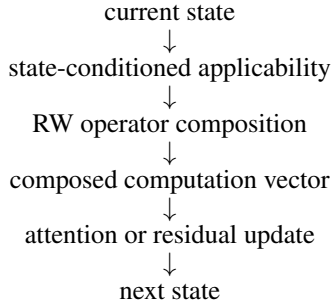
Split	Units	Intact margin	Suppressed margin	Drop (95% CI)	Accuracy	Recovery / restored acc.
Validation	123	3.465456	-4.399331	7.864787 [7.490080, 8.241263]	1.000000 → 0.008130	0.999661 / 0.991870
Held-out test	114	3.254720	-4.155613	7.410332 [7.016485, 7.818472]	1.000000 → 0.008772	0.998406 / 1.000000

Table 6. Fail-closed mechanistic results. Later phases were not opened after a failed preregistered gate.

Study	Passed	Failed boundary	Decision
RAVEL 400M	Behavioral eligibility	Variable stability: 0.628762 / 0.676243 / 0.660622	No frozen circuit; no validation, transfer, or test
ARC 400M	Stable discovery; 4,096 Q/K sites; restoration	Suppression effect wrong sign; control separation failed	Reject causal replication; held-out test closed
IOI 1.3B	Behavioral eligibility; rank stability 0.907188	4,096-site cumulative importance 0.635144 < 0.70	No frozen circuit; validation and test closed

whether localized compositions are causally involved in behavior. Semantic interpretation of these objects is a subsequent empirical question rather than a prerequisite of the architecture.

The central abstraction is a state-dependent map from the current representation to its applicable RW primitives, their continuously weighted composition, the resulting composed computation vector, and the state update it parameterizes:



The operator composition makes the intermediate computation explicit. For Q, K, and V, the composed vectors form the interfaces used by attention; for RST, the composed vector is applied directly as a residual-state update.

The operation-address space implements one step inside this abstraction. A query represents the current applicability condition, an address identifies a reusable primitive for that test, and the RW directions define the primitive’s executed function. Because the address is derived from read/write parameters, applicability geometry and RW function are coupled rather than assigned independent labels. This coupling is an architectural constraint, not proof of semantic interpretability. A primitive may be explicit, observable, and causally testable while participating in distributed or poly-semantic computation. The IOI circuit’s 4,096 sites across all layers is consistent with this distributed view. Its suppression and exact-restoration results show that the exposed contributions can serve as causal intervention targets; they do not show that the circuit is a human-readable algorithm.

Viewed as a research interface, this perspective decomposes a broader question about learned computation into several more explicit problems: how operations relevant to a representation state should be organized, how the current state should identify them, how multiple applicable operations should compose, and how the resulting conditional computation should be executed. These are not claims that a trained language model implements a discrete symbolic program. Rather, they define axes along which learned transition-producing computation can be made more explicit, structured, and experimentally testable. Making these objects explicit also provides computational handles for analysis and causal intervention, and motivates the longer-term question of whether learned computation can become reliably editable and composable rather than merely observable.

6.3. From Functional Conditionality to Structured Conditional Execution

The present results establish functional conditionality: for a given representation state, many operators have exactly zero applicability. They do not yet establish structural conditionality. Each state still evaluates globally visible type-specific operator pools before those zero contributions are known. A stronger form of conditional computation would organize operators so that the current state need only identify a substantially smaller candidate region before composition and execution.

This distinction opens several coupled questions. Operator organization may be flat or hierarchical, globally shared or locally partitioned; identification may use exact addressing or approximate retrieval; composition need not be restricted to the present normalized additive form; and physical execution requires dispatch mechanisms that avoid evaluating inactive candidates. The relevant empirical question is whether such restrictions can reduce computational visibility and executed computation while preserving language-model

quality.

One preliminary direction toward this goal is the *implemented but not yet fully evaluated low-dimensional operation-space extension*. The separate repository implementation uses low-dimensional addresses but full- D RW execution. It associates each operation-space address with $D \leftrightarrow R$ interfaces, selects spaces independently for Q, K, V, and RST, and performs local RW composition before writing back:

$$h_m = P_m^\top x, \quad (22)$$

$$z_{a,m} = \sum_i c_{a,m,i}(h_m) \langle h_m, r_{a,m,i} \rangle w_{a,m,i}, \quad (23)$$

$$\Delta x_a = \sum_m \alpha_{a,m}(x) B_m z_{a,m}. \quad (24)$$

Its local operator remains globally rank-1 because $B_m w_i r_i^\top P_m^\top = (B_m w_i)(P_m r_i)^\top$. Compilation, symbolic parameter counts, reduced JIT forward/backward, and gradient contracts have been checked, and limited early-training observations exist. There is no completed controlled language-model evaluation or realized efficiency measurement. The current reference path executes all spaces densely; candidate-restricted dispatch and an accelerator-native kernel remain future work. Appendix H records the boundary in more detail.

6.4. Limitations

The present results do not establish that transition-producing computation has been factorized into an efficient conditional computational structure. Although each state has strict-subset functional operator support, applicability is still evaluated against large globally visible type-specific pools. Performance retention may therefore depend partly on the representational capacity provided by a large distributed operator bank and global condition evaluation, rather than on a compact or intrinsically efficient decomposition of language-model computation.

Relatedly, the learned spherical address geometry is not shown to reflect an intrinsic or compressible organization of language operations. It may instead serve as a high-capacity substrate in which useful operators are distributed broadly enough for global state-conditioned access to recover the required transformations. The present study therefore does not establish whether comparable language-model expressivity can be retained when operator visibility, candidate evaluation, and executed computation are restricted substantially further.

The current implementation evaluates full address and operator pools densely. Exact functional support does not

imply current sparse execution, and even the exact-support accounting remains 2.1168 times the matched dense Transformer’s backbone accounting. No production latency, energy, or hardware advantage is measured.

The central causal evidence is confined to IOI and does not automatically generalize to every task. RAVEL, ARC, and the selected 1.3B IOI replication provide valid negative boundaries. The 400M and 1.3B checkpoints have different parameter and training-token budgets, so they do not define a controlled scaling curve. Operator-level causal units need not map one-to-one to human-readable concepts. The low-dimensional extension has not received a controlled evaluation. Finally, the present work does not establish language reasoning as universally discrete operator programs.

7. Conclusion

DAWN-SRW offers a view of language-model computation in which a central part of state evolution is formulated as state-conditioned composition of explicit reusable RW primitives. A representation state determines which RW primitives are applicable and to what continuous degree; their composition produces the interfaces used by attention or a direct residual-state update. The explicit separation of learned operations, continuous applicability, and composition supports the operational interpretation developed in this paper: transition-producing computation as continuous conditional execution over reusable learned operators. The evaluated v4172 realization computes applicability through learned operation-address mappings derived from RW function and reuses QK-, V-, and RST-specific primitive pools across depth. It retains measured language-model performance relative to a matched 400M dense control. A discovery-frozen Q/K-centered operator circuit is causally necessary under suppression and recoverable under exact restoration on independent validation and fully held-out IOI inputs. The model also shows exact input-conditioned functional support, and the frozen IOI circuit demonstrates that exposed operator contributions can serve as causal intervention targets. These findings support explicit state-conditioned RW operator composition as a viable and causally analyzable model of language-model computation; they do not establish human-readable primitive semantics, freely editable or transferable computation, benchmark superiority, or current hardware efficiency. More broadly, these results motivate studying learned transition-producing computation through how state-relevant operations are organized, identified, composed, and executed. Whether this perspective can preserve language-model expressivity while substantially reducing global operator visibility and executed computation remains an open question.

A. Exact Addressing and Composition Equations

This appendix restates the production computation as a compact reference. Raw read and write parameters are normalized in float32 before being cast for dense contractions:

$$\bar{r}_i = \frac{r_i}{\max(\|r_i\|_2, \epsilon)}, \quad \bar{w}_i = \frac{w_i}{\max(\|w_i\|_2, \epsilon)}.$$

One pair of model-level probes is shared across QK, V, and RST:

$$u_i = \text{unit}(\bar{r}_i^\top P_r), \quad v_i = \text{unit}(\bar{w}_i^\top P_w), \\ k_i = \text{unit}(u_i \odot v_i).$$

The query and threshold use separate direct projections:

$$\hat{q}_a = \text{unit}(xP_a + b_a), \\ \tau_a = -1 + 2\sigma(xu_a + \beta_a), \quad \rho_i = \hat{q}_a^\top k_i.$$

For linear_angular composition:

$$c_i = \text{clip}\left(\frac{\rho_i - \tau_a}{\max(1 - \tau_a, 10^{-4})}, 0, 1\right)$$

and padding rows are masked to zero. This is the single applicability coefficient used in RW composition. Let $M_a = \sum_i c_i$. The production composed output before causal attention or residual addition is

$$F_a(x) = \sqrt{\frac{D}{L}} \frac{\sum_i c_i \langle x, \bar{r}_i \rangle \bar{w}_i}{\max(M_a, 1)^{p_a}},$$

with $p_Q = p_K = 0.5$, $p_V = 1.0$, and $p_{RST} = 1.2$. Operator rows are sharded over the model axis; local numerators and admission mass are reduced across shards. Chunking changes the contraction schedule, not the equation.

B. Complete Model and Training Settings

The 400M DAWN-SRW checkpoint is the primary performance and mechanistic model. The dense 400M checkpoint is only a matched performance control. The 1.3B DAWN-SRW checkpoint is used for descriptive performance and one fail-closed IOI localization replication. The still-incomplete dense 1.3B run is excluded.

C. Benchmark Metrics and Protocol Details

C4. All reported C4 rows use the same packed validation artifact and `packed_c4_validation_v1` protocol. The capped view contains 19,531 sequences. Exactly 19,520 sequences form 610 complete global evaluation batches, producing 9,974,720 valid next-token targets; 11 incomplete-tail sequences are excluded before scoring.

Zero-shot. The protocol is `mamba_table3_zero_shot_v1`, using stock `lm-eval==0.4.2`, global batch 32, maximum length 512, length buckets 64/128/256/512, and zero few-shot examples. Evaluated split sizes are 5,153 for LAMBADA, 10,042 for HellaSwag, 1,838 for PIQA, 2,376 for ARC-Easy, 1,172 for ARC-Challenge, and 1,267 for WinoGrande. Task, dataset, tokenizer, and software contracts match exactly between the two 400M models.

D. IOI Localization and Intervention Details

D.1. Frozen discovery

Discovery admits 119 paired-correct independent units and ranks 456,697 sites. Split-half rank stability is 0.913978765. The frozen selection rule is the smallest prefix satisfying cumulative absolute importance at least 0.70 and split top- k overlap at least 0.95. The selected $k = 4096$ prefix has:

- cumulative absolute importance 0.719248713;
- 3,993 shared split sites, overlap 0.974853516, and Jaccard 0.950940700;
- Q/K/V/RST counts = 1744/2350/2/0;
- coverage of every layer from 0 through 17.

Validation and test do not recompute or alter this selection.

D.2. Suppression, controls, and restoration

For a correct candidate c and source candidate s , the primary score is

$$\mathcal{M} = \sum_{t \in c} \log p(t) - \sum_{t \in s} \log p(t).$$

Suppression zeros the frozen sites' execution numerator at specified token positions. It leaves admission mass and the production denominator unchanged. Controls use the same 4,096-site count and exclude all circuit sites. Activation matching conditions on layer, computation type, and the discovery mean pre-scale operator-output-norm quantile. Restoration executes a distinct intervention: after suppression, it adds back the exact selected same-example numerator under the unchanged denominator.

On validation, restoration returns the margin to 3.462791, leaving residual drop 0.002665. On held-out test, it returns the margin to 3.242905, leaving residual drop 0.011815. The respective recovery fractions are 0.999661155 (95% CI [0.998654544, 1.000584125]) and 0.998405645 (95% CI [0.997323737, 0.999385266]).

Table 7. Detailed settings from the registered training configurations and restored checkpoint manifests.

Setting	DAWN-SRW 400M	Dense 400M	DAWN-SRW 1.3B
Vocabulary	30,522	30,522	30,522
Residual width	2,304	1,280	4,096
Layers	18	18	24
Heads / head width	36 / 64	20 / 64	32 / 128
Maximum sequence length	512	512	512
Feed-forward width	—	5,120	—
Operation-address width	256	—	256
QK / V / RST pool	4,600 / 14,500 / 29,300	—	8,534 / 26,904 / 54,352
QK / V / RST chunks	1 / 1 / 4	—	1 / 2 / 8
QK / V / RST denominator power	0.5 / 1.0 / 1.2	—	0.5 / 1.0 / 1.2
Initial active targets	0.05 / 0.03 / 0.01	—	0.05 / 0.03 / 0.01
Training batch	1,024	1,024	448
Mesh (data $\times$ model)	16×2	16×2	16×2
Learning rate	3×10^{-4}	3×10^{-4}	2×10^{-4}
Warmup ratio	0.08	0.08	0.06
Weight decay	0.1	0.1	0.1
Tau learning-rate multiplier	0.001	—	0.001
Dropout / <code>router_dropout</code>	0 / 0	0 / —	0 / 0
Gradient checkpointing	yes	yes	yes
Seed	1	1	1

Table 8. Control effects and frozen-minus-control separation. Each control family contains 100 replicates. All paired permutation p -values are 0.0004997501 with 2,000 permutations.

Control	Validation drop	Frozen minus control (95% CI)	Test drop	Frozen minus control (95% CI)
Equal-count	0.070886	7.793902 [7.437851, 8.161572]	0.080276	7.330057 [6.922408, 7.716879]
Same-layer	0.039389	7.825399 [7.440629, 8.180288]	0.041073	7.369260 [6.977447, 7.754539]
Activation-matched	5.400490	2.464297 [2.076200, 2.832697]	5.472579	1.937754 [1.564818, 2.319706]
Route-frequency	0.219162	7.645626 [7.289512, 8.031625]	0.242665	7.167668 [6.779836, 7.545537]

E. Negative Mechanistic Results

RAVEL. Discovery uses 142 independent units. Rank stability is 0.628762 for Continent, 0.676243 for Country, and 0.660622 for Language, below the preregistered requirement. No prefix passes the complete discovery rule, no variable circuit is frozen, and validation, transfer, and test are not opened.

ARC. Discovery stability is 0.89386 and a 4,096-site Q/K-only circuit is frozen. On 51 validation units, intact margin is 1.735534 and suppressed margin is 2.819139, giving a frozen margin drop of -1.083605 (95% CI $[-1.472180, -0.690348]$). The effect has the wrong sign and fails control separation. Restoration recovery is 1.004083 (95% CI $[0.994587, 1.013652]$), but restoration alone does not pass the conjunction. Held-out test remains closed.

1.3B IOI. The scale replication admits 125 independent units and passes rank stability at 0.907187749. The largest allowed 4,096-site prefix reaches only 0.635144 cumulative absolute importance, below the 0.70 gate. No circuit is frozen; validation and test remain closed.

F. Compute Accounting

The support profile evaluates 9,994,240 packed-C4 positions and observes address-stage margins before RW application. It stores aggregate statistics, not per-token operator identities or vectors. Static accounting uses batch 1, sequence

length 512, and 1 MAC = 2 FLOPs. It counts major matmul/einsum operations and omits softmax, normalization, gate elementwise arithmetic, embedding lookup, residual additions, and communication collectives.

The exact-support estimate is mathematically output-equivalent if the executor can skip all and only zero-weight RW terms. It does not include indexed address retrieval and does not demonstrate kernel parity. Instrumented profiling time is excluded from latency claims because histogram collection is part of the measured path.

G. Reproducibility Identities

The machine-readable checkpoint registry is `configs/paper_checkpoint_registry.yaml`. Exact evaluation commits, software versions, data hashes, result roots, and complete artifact hashes are recorded in the repository experiment ledger. Historical experiment branches are evidence identities only; this source is maintained on the canonical main branch.

H. Low-Dimensional Operation-Space Extension

The low-dimensional extension uses one shared atlas of $M = 8$ operation-space addresses, shared $D \rightarrow R$ read interfaces P_m , and shared $R \rightarrow D$ write interfaces B_m . Q, K, V, and RST have separate query projections, tau maps, and operator banks, and independently select two atlas entries.

Table 9. Backbone static accounting. “Exact support” retains full dense address scoring and replaces only full-pool RW application with the measured positive-margin support.

Component	Current dense DAWN-SRW	Dense address + exact RW	Dense Transformer
Address scoring + query/tau projections	0.293742	0.293742	–
RW application	4.306187	0.383157	–
Causal attention	0.043487	0.043487	0.024159
Dense output / QKV / FFN maps	0.097845	0.097845	0.362388
Total TFLOPs	4.741260	0.818230	0.386547

Table 10. Primary immutable identities. Branch names are omitted because they move; exact checkpoint steps, hashes, and artifact locations are the binding identities.

Evidence	Exact identity	Repository source
DAWN-SRW 400M	model spatial-r1-v4.1.7.2; step 76,293; checkpoint identity a7ce8afc...f91c17; config hash 08733ae4...5039b	configs/train_config_v4172_400M_c4_40B_v4_64_ver1_den_qk0p5_vlp0_rst1p2.yaml
Dense 400M	model baseline-JAX; step 76,293; config hash c3ce5d72...82fc25	configs/train_config_baseline_tpu_400M_c4_40B_v4_64_tp2.yaml
DAWN-SRW 1.3B	model spatial-r1-v4.1.7.2; step 87,193; checkpoint identity 2e38b41f...bf5f9; config hash 1e877263...4a5222	configs/train_config_v4172_1p3B_c4_20B_v4_64_ver1_den_qk0p5_vlp0_rst1p2.yaml
IOI circuit	parsed frozen-spec hash 6ec26618...418c; site hash 573cfff8...0200; selected-row hash 1439f6f3...fe0c1	configs/paper_ioi_frozen_circuit_v4172_400m.yaml
Compute support	support hash bleac5f5...bdc2; accounting hash d7ad6115...984e	assets/paper/v4172_400m_compute_support_step76293/

For local operator i in space m ,

$$\begin{aligned}\mathcal{O}_{a,m,i}^{\text{global}}(x) &= B_m w_{a,m,i} r_{a,m,i}^\top P_m^\top x \\ &= (B_m w_{a,m,i})(P_m r_{a,m,i})^\top x,\end{aligned}$$

so the induced global transformation remains rank-1.

The 40M reference has 40,341,088 parameters, $D = 432$, $R = 128$, 12 layers, six heads, and top-2 space selection. Repository-local compilation, abstract-tree and symbolic parameter counting, shared-key and independent Q/K/V/RST space selection, legacy-checkpoint rejection, reduced JIT forward/backward, selector metrics, and gradient contracts have passed. Some early-training observations are recorded, but the exact deployment and complete controlled language-model evaluation are absent. No latency, throughput, energy, or hardware-efficiency result exists. The reference path executes every space densely; future work requires candidate-restricted dispatch and an accelerator-native execution kernel.

Acknowledgments

The author thanks Anthropic Claude, Claude Code, OpenAI ChatGPT, and OpenAI Codex for assistance throughout this research. This work was developed with extensive use of AI systems for literature exploration, conceptual exploration, cross-domain framing, abstraction and terminology refinement, implementation support, debugging, and manuscript preparation. In particular, these systems were used as interactive assistants for refining the learned state-transition formulation and the exposition of DAWN-SRW. The research direction, architectural decisions, experimental design, experiments, analyses, interpretations, and final claims remain the responsibility of the author.

This research was supported by the Google TPU Research Cloud (TRC) program, which provided computational resources for training and analysis.

References

- Berges, V.-P., Oğuz, B., Haziza, D., Yih, W.-t., Zettlemoyer, L., and Ghosh, G. Memory layers at scale. *arXiv preprint arXiv:2412.09764*, 2024. doi: 10.48550/arXiv.2412.09764.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Geva, M., Schuster, R., Berant, J., and Levy, O. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5484–5495, 2021. doi: 10.18653/v1/2021.emnlp-main.446.
- He, X. O. Mixture of a million experts. *arXiv preprint arXiv:2407.04153*, 2024. doi: 10.48550/arXiv.2407.04153.
- Huang, J., Wu, Z., Potts, C., Geva, M., and Geiger, A. RAVEL: Evaluating interpretability methods on disentangling language model representations. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Kramár, J., Lieberum, T., Shah, R., and Nanda, N. AtP*: An efficient and scalable method for localizing LLM behaviour to components. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Lu, H., Zhao, C., Xue, M., Yao, L., Moore, K., and Gong, D. Little by little: Continual learning via incremental mixture of rank-1 associative memory experts. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026.
- Neelakantan, A., Le, Q. V., and Sutskever, I. Neural programmer: Inducing latent programs with gradient descent. In *International Conference on Learning Representations*, 2016.
- Plotkin, G. D. A structural approach to operational semantics.

- Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, Aarhus, Denmark, September 1981.
- Reed, S. and de Freitas, N. Neural programmer-interpreters. In *International Conference on Learning Representations*, 2016.
- Schlag, I., Irie, K., and Schmidhuber, J. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9355–9366, 2021.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.
- Shi, J., Peng, Z., Zhu, Y., Wu, Y., Liu, G., and Luo, Y. OmniMoE: An efficient MoE by orchestrating atomic experts at scale. *arXiv preprint arXiv:2602.05711*, 2026. doi: 10.48550/arXiv.2602.05711.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Wang, K., Variengien, A., Conmy, A., Shlegeris, B., and Steinhardt, J. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations*, 2023.